

FSM para Controle de Ponte Estreita com Prioridade de Acesso

Natan Ramos Barbosa
Engenharia Eletrônica
Universidade de Brasília (UnB)
140029371@aluno.unb.br

Noboru Luiz Watanabe Monteiro
Engenharia Eletrônica
Universidade de Brasília (UnB)
231026527@aluno.unb.br

Abstract—Este trabalho visa demonstrar o funcionamento teórico e prático de uma Máquina de Estados Finitos (FSM) aplicada ao controle de uma ponte estreita. A FSM projetada possui sete estados, prioriza o tráfego vindo do lado esquerdo da ponte e impede o uso simultâneo por dois veículos. A implementação foi realizada em VHDL, com simulação no Vivado e testes na FPGA Basys 3.

Index Terms—FSM, VHDL, FPGA, controle de tráfego, ponte estreita

I. ESPECIFICAÇÃO

O sistema detecta veículos pelas entradas D1 a D6. Quando os sensores de entrada (D1, D5) são ativados, os sinais (S1, S2) e cancelas (C1, C2) são abertos. Após a detecção nos sensores intermediários (D2, D3), os acessos são fechados. O sistema libera novas entradas somente após a saída dos veículos (D4, D6).

Espera-se que o sistema retenha qualquer veículo que tente iniciar uma travessia enquanto outra está em progresso. Dessa forma, mesmo que uma cancela se abra, a sinalização com semáforos indicará quando o(a) condutor(a) poderá iniciar sua passagem. De forma semelhante, para evitar qualquer situação em que dois veículos transitem simultaneamente, em caso de dois veículos se posicionarem frente aos sensores ao mesmo tempo, o(a) condutor(a) que trafegar pelo sentido D1-D4 terá prioridade. Ao mesmo tempo, por motivos de viabilidade técnica e aplicabilidade, não há mecanismo que impeça o(a) condutor(a) de entrar pelo sensor D1 e sair pelo sensor D6 pela mesma cancela C1, ou entrar pelo sensor D5 e sair pelo sensor D4 pela cancela C2, caso seja necessário.

Ferramentas utilizadas:

- Diagrama de estados
- Tabela de transição
- Código em VHDL
- FPGA Basys 3

Recursos da placa FPGA (Basys 3) utilizados:

- Look-up-tables: 15 (0.07%)
- Flip-flops: 7 (0.02%)
- Pinos IO: 15 (14.15%)
- Buffers: 1 (3.13%)

Trabalho apresentado na disciplina de Prática de Circuitos Digitais 2, Universidade de Brasília.

II. PROCEDIMENTOS

A Figura 1 ilustra o modelo da ponte estreita.

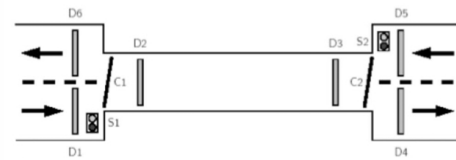


Fig. 1. Acesso a uma estrada estreita de montanha (ilustrativo)

A implementação em VHDL seguiu o seguinte diagrama de estados

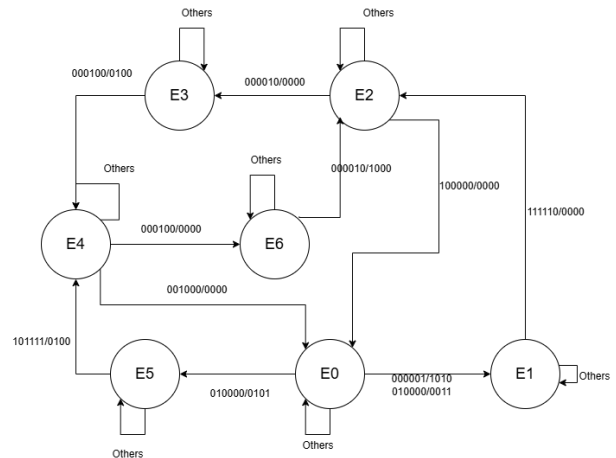


Fig. 2. Diagrama de Transição de Estados

A. Interpretação do Problema

Cada um dos integrantes do grupo realizou a leitura do roteiro, disponibilizado pelo professor na plataforma *Notion*, e realizou um diagrama de estados e esboço do projeto de automação solicitado.

B. Protótipos Iniciais

Foram projetadas duas versões iniciais de protótipo para a resolução do problema, como atividade de preparação. Esses protótipos incluíram um arquivo VHD para programar a *FPGA* e um arquivo *testbench* para simulação em *software*.

C. Elaboração do Código

Inicialmente, o sistema foi modelado como uma máquina de 6 estados distintos e as seguintes entradas e saídas:

- Entradas: D1, D2, D3, D4, D5 D6 (standard logic)
- Saídas: S1, S2, C1, C2 (standard logic)

Devido a erros de compilação e dificuldade de manipulação, as entradas foram substituídas por um vetor de 6 bits contendo todos os detectores da estrada. E as saídas por um vetor de 4 bits de forma análoga às entradas.

- Entradas: D(5 downto 0) (standard logic vector)
- Saídas: leds(3 downto 0) (standard logic vector)

D. Simulações e Testes

Assim, possível realizar as primeiras simulações utilizando a ferramenta *testbench* do *software Vivado*, mais detalhes serão descritos na seção seguinte.

Contudo, os testes realizados com o código implementado na placa revelaram uma falha no funcionamento da máquina. O mal funcionamento se deu pela falta de estados que determinassem o sentido dos veículos quando estes estivessem atravessando a pista. Isso significa que, com aquela máquina, um veículo que estivesse passando pelo sensor D2(1) no sentido D1-D4, ao transitar de estado, poderia ficar preso entre as cancelas da pista, pois a máquina interpretaria que um veículo estava de saída pelo sentido D5-D6.

Assim, foi incrementado um estado adicional no projeto final da *FSM* e mais um teste com código implementado foi realizado. O código em sua versão final foi testado sob a supervisão do professor e obteve êxito em funcionar sob situações de possível ambiguidade.

III. IMPLEMENTAÇÃO EM VHDL

A. Definição de Estados

Na Arquitetura do sistema, foi feita a declaração de variáveis do tipo 'type' para os Estados possíveis, na qual posteriormente foram sinalizadas outras duas variáveis do tipo Type 'estado atual' e 'próximo estado' ambas inicializadas no Estado "e0" para configurar a máquina de estados com os estados futuros:

```
type tipo_estado is (e0, e1, e2, e3,
                    e4, e5, e6);
signal estado_atual,
        proximo_estado : tipo_estado := e0;
```

- E0: Tudo fechado, pista vazia
- E1: C1 e S1 abertos
- E2: C2 aberta, S1 fechado
- E3: Tudo fechado, pista ocupada (esquerda-direita)
- E4: C2 aberta, S2 fechado
- E5: C2 e S2 abertos
- E6: Estado auxiliar/reservado

B. Entidade

```
entity semaforo_sensores is
Port ( btnR : in STD_LOGIC;
      btnC : in STD_LOGIC;
      sw : in STD_LOGIC_VECTOR (5 downto 0);
      led : out STD_LOGIC_VECTOR (3 downto 0);
      mostrar_estado : out STD_LOGIC_VECTOR (2 downto 0)
end semaforo_sensores;
```

Sendo a variável "btnR" funcionando como um clock assíncrono para a mudança de estados na borda do clock, o "btnC" funcionando como um reset que devolve os sistema para o estado inicial e0, a variável "sw" o vetor das entradas dos sensores, a variável "led" o vetor com as saídas dos semáforos e cancelas e a "mostrar estado" como uma variável de saída que mostra em qual estado o sistema está.

C. Comportamento da Arquitetura

1. Registro de Estado: Atualiza o estado na borda de subida de btnR. Se btnC = '1', reseta para e0.

2. Transição de Estado: Define o próximo estado conforme as entradas:

- e0: sw = "000001" ou "010001" → e1; sw = "010000" → e5
- e1: sw(0) = '0' → e2
- e2: sw(1) = '1' → e3; sw(5) = '1' → e0
- e3: sw(2) = '1' → e4
- e4: sw(3) = '1' → e0; sw(2) = '1' → e6
- e5: sw(4) = '0' → e4
- e6: sw(1) = '1' → e2

D. Simulação em Testbench

O trecho abaixo descreve os estímulos gerados aos sinais da máquina de estados, para simulação:

```
sclk <= not sclk after 5ns; --Tclk = 10ns
srst <= '0', '1' after 15ns,
        '0' after 25ns,
        '1' after 200ns,
        '0' after 205ns;

ssw <= "000000",
        "000001" after 25ns,
        "000000" after 35ns,
        "000010" after 45ns,
        "000110" after 55ns,
        "001000" after 65ns,
        "000000" after 75ns,
        "010001" after 85ns,
        "000000" after 95ns,
        "100010" after 105ns,
        "110011" after 115ns,
        "000100" after 120ns,
        "001000" after 125ns,
        "010000" after 135ns,
        "000000" after 145ns,
        "000100" after 155ns,
        "000010" after 165ns,
```



Fig. 3. Simulação das entradas via *testbench*

```
"100000" after 175ns,  
"010001" after 185ns,  
"000000" after 200ns;
```

Em 15 nanossegundos, é acionado o *reset*. Então, após este ser desligado, inicia-se a simulação de um veículo atravessando em sentido D1-D4. Em 55 nanossegundos, os sensores D2 e D3 são ligados simultaneamente no mesmo estado e instante de tempo, para testar a robustez do sistema. Em 85 nanossegundos, as entradas D1 e D5 são ligadas simultaneamente a fim de testar o sistema de prioridades da máquina. Em 105 nanossegundos, as entradas D6 e D2 são ligadas simultaneamente para teste de robustez. A partir de 135 nanossegundos, inicia-se o processo de veículo atravessando em sentido D5-D6. Por fim, em 200 nanossegundos é acionado o *reset* para que o circuito retorne ao seu estado inicial.

IV. RESULTADOS E DISCUSSÃO

O sistema demonstrou funcionalidade conforme esperado. A prioridade de passagem foi corretamente atribuída à entrada esquerda (D1). Nos estados e3 e e5, a FSM impede novas mudanças até que os sensores de saída sejam ativados, garantindo consistência com o problema proposto.

V. CONCLUSÃO

Com uma FSM de sete estados foi possível implementar com sucesso o controle de tráfego em uma ponte estreita, respeitando a prioridade de sentido e evitando o uso simultâneo da via. A FPGA Basys 3 foi eficaz na execução e testes com clock assíncrono.

REFERÊNCIAS

REFERENCES

- [1] IEEE, "Conference Paper Templates," [Online]. Available: <https://www.ieee.org/conferences/publishing/templates.html>
- [2] "Your connected workspace for wiki, docs & projects — Notion," Notion, 2025. <https://gsbeserra.notion.site/Pr-tica-de-Eletr-nica-Digital-2-1-2025-1bf745480f10803e9dfbd8ec54e7002b?p=1bf745480f10811289bee5f27fc86bf7&pm=s> (accessed May 18, 2025).